C++ documentation :

- **C++ is a general-purpose programming language**
- **Developed by Bjarne Stroustrup**
- **Extension of C language**
- **Supports both:**
    - **procedural programming**
    - **Object-Oriented Programming (OOP)**

Uses of C++ :

- Software development
- Game development
- Competitive programming
- Operating systems
- Embedded systems

2. Basic Structure of a C++ Program :

```
#include <iostream>
using namespace std;

int main() {
   cout << "Hello World";
   return 0;
}
```

Explanation:

- *#include <iostream> ➛ Header file for input/output*
- *using namespace std; ➛ Avoids writing std:: again and again*
- *int main() ➛ Execution starts from here*
- *cout ➛ Output*
- *return 0; ➛ Ends the program*

3. Data Types in C++ :

Data Type	Size	Example
int	4 bytes	10
float	4 bytes	3.14
double	8 bytes	3.14159
char	1 byte	'A'
bool	1 byte	true / false
string	—	"Hello"

4. Variables :

- A **variable** is used to store data
- Syntax:

data_type variable_name;

Example:

```
int age = 18;
float marks = 85.5;
char grade = 'A';
```

---

5. Input and Output :

Output using cout

```
cout << "Welcome to C++";
```

Input using cin

```
int x;
cin >> x;
```

---

6. Control Statements :

6.1 if Statement

```
if (a > b) {
   cout << "A is greater";
}
```

6.2 if-else Statement

```
if (a % 2 == 0) {
   cout << "Even";
} else {
   cout << "Odd";
}
```

---

7. Loops :

7.1 for Loop

```
for (int i = 1; i <= 5; i++) {
   cout << i << endl;
}
```

7.2 while Loop

```
int i = 1;
while (i <= 5) {
   cout << i << endl;
   i++;
}
```

7.3 do-while Loop

```
int i = 1;
do {
   cout << i << endl;
   i++;
} while (i <= 5);
```

---

8. Functions :

- A **function** is a block of code that performs a specific task
- Helps in **code reusability**

Function Example:

```
void greet() {
   cout << "Hello";
}

int main() {
   greet();
   return 0;
}
```

---

9. Arrays :

- Used to store multiple values of the same type

Example:

```
int arr[5] = {1, 2, 3, 4, 5};
```

Accessing Elements:

```
cout << arr[0];   // First element
```

---

10. Strings :

```
#include <string>
string name = "Mansi";
cout << name;
```

---

11. Pointers :

- A **pointer** stores the address of another variable

```
int x = 10;
int *p = &x;
cout << p;
```

---

12. Object-Oriented Programming :

12.1 Class and Object

```
class Student {
public:
   int roll;
   string name;
};

int main() {
   Student s1;
   s1.roll = 1;
   s1.name = "Rahul";
}
```

---

13. Important Keywords :

Keyword	Use
return	Sends value back
void	No return value
break	Exit loop
continue	Skip iteration
const	Constant value

time and space complexity

In this particular theoretical part we see that some code blocks take more space and time to run and give output thus time and space complexity is the amount of space and time the particular code block takes to run and give output.

Less time complexity and space complexity : better and faster code .

Even odd example : if (n%2 = 0)
cout<< “even”
Else
Cout << “odd”
This particular code takes a longer time to run.

But the code here has less time and space complexity :

Now when we write the code what happens is c++ is a compiler and it reads all the code block altogether that is it is a compiler so it reads the sentence altogether so the compiler saves it as tokens :

For example : if (n%2=0)
If is the keyword here
The brackets () act as token
And even the semicolon acts as a token
SO IF ANY TOKEN IS MISSING THE CODE THROWS AND ERROR .

Things taken in class today :

1. Flowchart :

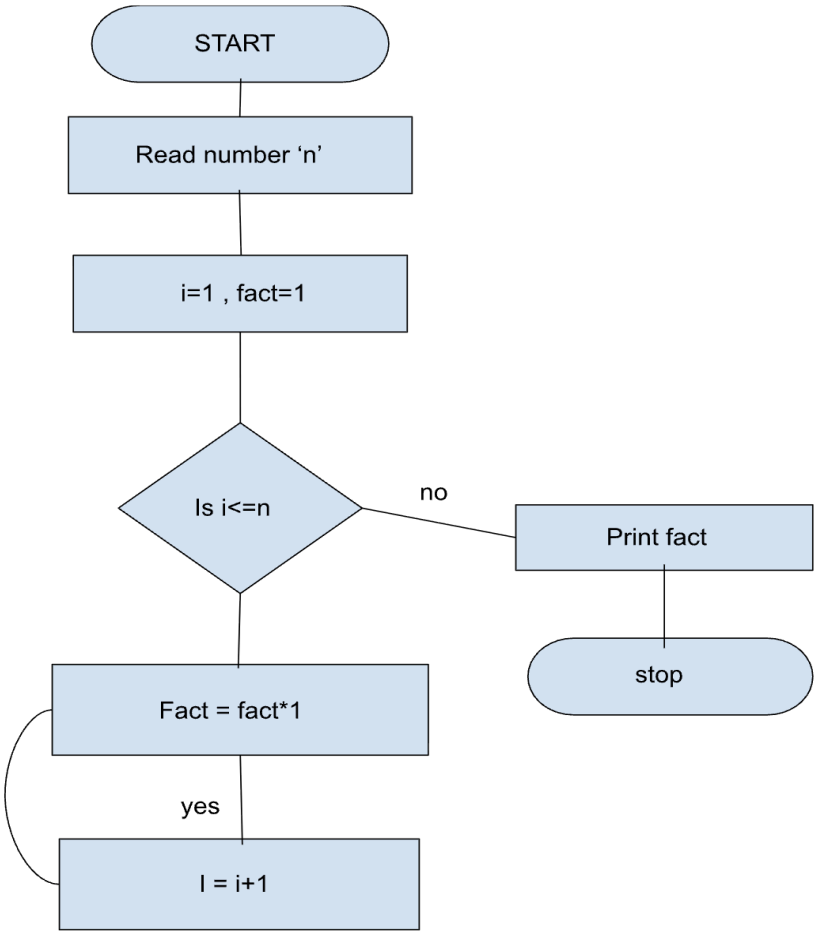
Oval box => oval button
Statements => rectangular box : code key statements
Conditions => diamond box
Segregation => arrows

2. Tokens :

About how the codes are written and how the compiler is used.

range.cpp , factorial.cpp and sumof.cpp

FLOWCHART EXAMPLES :



4.RUNNING OF CODE IN REVERSE :

In this particular code we intend to code the most basic format code block using which we can print the numbers in reverse given that we haven't learnt the data structures yet . thus we use **POSITION to reverse the code .**

FUNCTIONS IN C++ :

---

1. What is a Function?

A **function** is a block of code that:

- Performs a **specific tasks**
- Can be **reused**
- Makes programs **short, readable, and organized**

Instead of writing the same code again and again, we put it inside a function.

---

2. Why Do We Use Functions?

- Code reusability
- Easy debugging
- Better readability
- Modular programming

---

3. Types of Functions in C++ :

3.1 Built-in Functions

Predefined in C++

Examples:

```
sqrt(25);
pow(2,3);
```

3.2 User-defined Functions

Created by the programmer

Example:

```
void greet() {
   cout << "Hello";
}
```

---

4. Parts of a Function

Every function has **three main parts**:

1 Function Declaration (Prototype)

Tells the compiler:

- Function name
- Return type
- Parameters

```
int add(int, int);
```

---

2 Function Definition

Actual code of the function

```
int add(int a, int b) {
   return a + b;
}
```

---

3 Function Call

Calling the function in main()

```
int sum = add(10, 20);
```

---

5. Syntax of a Function :

```
return_type function_name(parameters) {
   // function body
   return value;
}
```

---

6. Types of Functions Based on Arguments & Return Value :

Type 1: No Arguments, No Return Value

```
void display() {
   cout << "Welcome";
}

int main() {
   display();
}
```

Used when output is directly printed

Type 2: Arguments, No Return Value

```
void add(int a, int b) {
   cout << a + b;
}

int main() {
   add(3, 4);
}
```

Input is passed, but no value is returned

Type 3: No Arguments, Return Value

```
int getNumber() {
   return 10;
}

int main() {
   int x = getNumber();
}
```

Function returns a value without input

Type 4: Arguments and Return Value

```
int add(int a, int b) {
   return a + b;
}

int main() {
   int sum = add(5, 6);
}
```

Most commonly used in real programs

---

7. Function with Multiple Parameters

```
int max(int a, int b, int c) {
   if (a > b && a > c)
      return a;
   else if (b > c)
      return b;
   else
      return c};
```

8. Call by Value

- A **copy** of the variable is passed
- Original value does NOT change

```
void change(int x) {
   x = 10;
}

int main() {
   int a = 5;
   change(a);
   cout << a;   // Output: 5
}
```

---

9. Call by Reference

- Address of variable is passed
- Original value **changes**

```
void change(int &x) {
   x = 10;
}

int main() {
   int a = 5;
   change(a);
   cout << a;   // Output: 10
}
```

---

10. Inline Functions

- Function code replaces the function call
- Used for **small functions**
- Improves speed

```
inline int square(int x) {
   return x * x;}
```

---

11. Recursive Functions

A function that **calls itself**

Example: Factorial

```
int fact(int n) {
   if (n == 0)
      return 1;
   else
      return n * fact(n - 1);
}
```

Imp: Must have a base condition

---

12. Function Overloading

- Same function name
- Different parameters

```
int add(int a, int b) {
   return a + b;
}

double add(double a, double b) {
   return a + b;
}
```

Compile-time polymorphism

---

13. Advantages of Functions

- Reduces code length
- Avoids repetition
- Easy maintenance
- Better understanding

14.Short Notes

- Function = group of statements
- Function improves modularity
- void ⇒ no return
- Call by value ⇒ copy
- Call by reference ⇒ address

Structure programing

1. DATA TYPES (C Language)

What is a Data Type?

A **data type** specifies:

- What kind of data a variable can store
- How much memory it takes
- What operations can be performed

Types of Data Types in C

1. Primitive (Basic) Data Types

Data Type	Description	Example
int	Stores integers	10, -5
float	Stores decimal values	3.14
double	Stores large decimal values	123.456
char	Stores single character	'A'

Example Code

```
#include <stdio.h>
int main() {
   int a = 10;
   float b = 3.5;
   char c = 'X';
   printf("%d\n", a);
   printf("%f\n", b);
   printf("%c\n", c);
```

```
 return 0;
}
```

---

2. OPERATORS

What are Operators?

Operators are **symbols used to perform operations** on variables and values.

Types of Operators

1. Arithmetic Operators

Operator	Meaning
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

```
int a = 10, b = 3;
printf("%d", a + b);
```

---

2. Relational Operators

Operator	Meaning
>	Greater than
<	Less than
==	Equal to
!=	Not equal

```
if (a > b)
    printf("a is greater");
```

---

3. Logical Operators

Operator	Meaning
&&	AND
`	
!	NOT

```
if (a > 5 && b < 5)
    printf("True");
```

---

3. CONDITIONAL STATEMENTS

Conditional statements are used to **make decisions**.

A. IF – ELSE STATEMENT

Syntax

```
if (condition) {
   statement;
} else {
   statement;
}
```

Example

```
#include <stdio.h>
int main() {
   int age = 18;

   if (age >= 18)
      printf("Eligible to vote");
   else
      printf("Not eligible");

   return 0;
}
```

Flowchart (If–Else)

```
    Start
      |
   Condition?
    /     \
  Yes     No
   |       |
Statement Statement
     \   /
      End
```

B. SWITCH CASE

Syntax

```
switch(expression) {
   case value1:
      statement;
      break;
   case value2:
      statement;
      break;
   default:
      statement;
}
```

Example

```
#include <stdio.h>
int main() {
   int day = 2;
   switch(day) {
      case 1: printf("Monday"); break;
      case 2: printf("Tuesday"); break;
      default: printf("Invalid day");
   }
   return 0;
}
```

Flowchart (Switch Case)

```
      Start
        |
   Expression
   /   |    \
Case1 Case2 Default
  |     |      |
Stmt  Stmt   Stmt
        |
       End
```

---

4. LOOPS

Loops are used to **repeat a block of code**.

A. FOR LOOP

Syntax

```
for(initialization; condition; increment) {
   statement;
}
```

Example

```
#include <stdio.h>
int main() {
   for(int i = 1; i <= 5; i++) {
      printf("%d\n", i);
   }
   return 0;}
```

Flowchart (For Loop)

```
 Start
   |
Initialize
   |
Condition?
  /   \
 Yes   No
  |     |
Statement
  |
Increment
  |
Condition?
   |
  End
```

B. WHILE LOOP

Syntax

```
while(condition) {
   statement;
}
```

Example

```
int i = 1;
while(i <= 5) {
   printf("%d\n", i);
   i++;
}
```

Flowchart (While Loop)

```
 Start
   |
Condition?
  /   \
 Yes   No
  |     |
Statement
  |
Back to Condition
   |
  End
```

C. DO–WHILE LOOP

Syntax

```
do {
   statement;
} while(condition);
```

Example

```
int i = 1;
do {
   printf("%d\n", i);
   i++;
} while(i <= 5);
```

Flowchart (Do–While)

```
 Start
   |
Statement
   |
Condition?
  /   \
 Yes   No
  |     |
Repeat  End
```

OOPS (OBJECT ORIENTED PROGRAMMING SYSTEM)

What is OOPS?

OOPS is a programming approach based on **objects and classes**.
It helps to write **secure, reusable, and easy-to-maintain code**.

Main Features of OOPS

- Class
- Object
- Inheritance
- Encapsulation
- Abstraction

- Polymorphism

---

1. CLASS

Definition

A **class** is a **blueprint or template** used to create objects.
It contains:

- **Data members (variables)**
- **Member functions (methods)**

Highlights

Logical entity
Does not occupy memory
Used to define structure of objects

Syntax

```
class ClassName {
   data members;
   member functions;
};
```

Example Program

```
#include <iostream>
```

```
using namespace std;

class Student {
public:
    int roll;
    void show() {
        cout << "Roll No: " << roll;
    }
};

int main() {
    Student s;
    s.roll = 10;
    s.show();
    return 0;
}
```

---

2. OBJECT

Definition

An **object** is an **instance of a class**.
It represents a **real-world entity**.

Highlights

Physical entity
Occupies memory
Created using class

Example

```
Student s1;   // s1 is an object
```

Example Program

```
#include <iostream>
using namespace std;

class Car {
public:
   void run() {
      cout << "Car is running";
   }
};

int main() {
   Car c1;
   c1.run();
   return 0;
}
```

---

3. INHERITANCE

Definition

Inheritance allows one class to **acquire properties of another class**.

Highlights

Improves code reusability
Saves time
Supports hierarchical classification

Types of Inheritance

- Single
- Multiple
- Multilevel
- Hierarchical
- Hybrid

Syntax

```
class Child : public Parent {
};
```

Example Program:-

```
#include <iostream>
using namespace std;

class Parent {
public:
   void show() {
      cout << "Parent class";
   }
};

class Child : public Parent {
};

int main() {
   Child c;
   c.show();
   return o;}
```

Diagram

```
Parent
  |
 Child
```

---

4. ABSTRACTION (ABSTRACT CLASS)

Definition

Abstraction hides **internal implementation** and shows **only essential details**.

Highlights

Improves security
Reduces complexity
Achieved using abstract class

Abstract Class

- Contains **at least one pure virtual function**
- Object **cannot** be created

Syntax

```
virtual void show() = 0;
```

Example Program

```
#include <iostream>
using namespace std;
```

```
class Shape {
public:
   virtual void draw() = 0;  // Pure virtual function
};

class Circle : public Shape {
public:
   void draw() {
      cout << "Drawing Circle";
   }
};

int main() {
   Circle c;
   c.draw();
   return 0;
}
```

5.ENCAPSULATION

Definition

Encapsulation means **binding data and methods together** into a single unit (class).

Highlights

Data hiding
Increases security
Controlled access using getters/setters

Example Program

```
#include <iostream>
using namespace std;

class Account {
private:
   int balance;

public:
   void setBalance(int b) {
      balance = b;
   }
   int getBalance() {
      return balance;
   }
};

int main() {
   Account a;
   a.setBalance(5000);
   cout << a.getBalance();
   return 0;
}
```

Diagram

```
+----------------+
|   Account      |
|----------------|
| - balance      |
|----------------|
| + setBalance() |
| + getBalance() |
+----------------+
```

DIFFERENCE TABLE (VERY IMPORTANT FOR EXAMS)

Feature	Class	Object
Type	Blueprint	Instance
Memory	No	Yes
Creation	Once	Multiple

ONE-LINE REVISION POINTS

1. Class ↠ Blueprint
2. Object ↠ Real world entity
3. Inheritance ↠ Reuse code
4. Abstraction ↠ Hide implementation
5. Encapsulation ↠ Data security

ACCESS SPECIFIERS (C++)

Access specifiers define **how class members can be accessed**.

1. PUBLIC

- Members are accessible **from anywhere**
- Used when data needs to be open

Example

```
#include <iostream>
using namespace std;

class Student {
public:
   int roll;
};

int main() {
   Student s;
   s.roll = 10;
   cout << s.roll;
   return 0;
}
```

---

2. PRIVATE

- Members are accessible **only inside the class**
- Used for **data hiding**

Example

```
#include <iostream>
```

```
using namespace std;

class Student {
private:
    int roll;
public:
    void setRoll(int r) {
        roll = r;
    }
    void showRoll() {
        cout << roll;
    }
};

int main() {
    Student s;
    s.setRoll(5);
    s.showRoll();
    return 0;
}
```

---

3. PROTECTED

- Accessible **inside class and derived (child) class**
- Used in inheritance

Example

```
#include <iostream>
```

```
using namespace std;

class A {
protected:
    int x = 10;
};

class B : public A {
public:
    void show() {
        cout << x;
    }
};

int main() {
    B obj;
    obj.show();
    return 0;
}
```

---

1. DATA TYPES

Data types specify the **type of data** stored in a variable.

Common Data Types

Data Type	Meaning	Example

int	Integer	10
float	Decimal	2.5
double	Large decimal	12.45
char	Character	'A'

Program

```
#include <stdio.h>
int main() {
   int a = 5;
   float b = 2.5;
   char c = 'A';

   printf("%d %f %c", a, b, c);
   return 0;
}
```

---

2. ARITHMETIC OPERATORS

Used for **mathematical calculations**.

Operator	**Use**

+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Program

```
int a = 10, b = 3;
printf("%d\n", a+b);
printf("%d\n", a-b);
printf("%d\n", a*b);
printf("%d\n", a/b);
printf("%d\n", a%b);
```

---

3. LOGICAL OPERATORS

Used to combine **conditions**.

Operator	Meaning
&&	AND
!	NOT

Program

```
int a = 10;
if(a > 5 && a < 20)
   printf("True");
```

---

4. CONDITIONAL OPERATORS (Ternary Operator)

Used as a **short form of if–else**.

Syntax

condition ? statement1 : statement2;

Example

```
int a = 10;
(a % 2 == 0) ? printf("Even") : printf("Odd");
```

---

5. IF–ELSE ➛ EVEN / ODD

Program

```
#include <stdio.h>
int main() {
   int num;
   printf("Enter number: ");
   scanf("%d", &num);

   if(num % 2 == 0)
      printf("Even number");
   else
      printf("Odd number");

   return 0;
}
```

Flowchart

```
 Start
   |
Input number
   |
num % 2 == 0 ?
  /      \
Yes      No
 |        |
Even     Odd
   \    /
     End
```

---

6. FACTORIAL PROGRAM

Factorial of n = n × (n−1) × (n−2) ... 1

Example (Using Loop)

```
#include <stdio.h>
int main() {
   int n, fact = 1;
   printf("Enter number: ");
   scanf("%d", &n);

   for(int i = 1; i <= n; i++) {
      fact = fact * i;
   }

   printf("Factorial = %d", fact);
   return 0;
}
```

---

7. FIBONACCI SERIES

Series: 0 1 1 2 3 5 8 ...

Program

```
#include <stdio.h>
int main() {
   int n, a = 0, b = 1, c;
   printf("Enter limit: ");
   scanf("%d", &n);

   for(int i = 1; i <= n; i++) {
      printf("%d ", a);
      c = a + b;
      a = b;
      b = c;
   }
   return 0;
```

}

---

8. REVERSE OF NAME (STRING)

Program

```
#include <stdio.h>
#include <string.h>

int main() {
    char name[20];
    printf("Enter name: ");
    gets(name);

    int len = strlen(name);
    for(int i = len-1; i >= 0; i--) {
        printf("%c", name[i]);
    }
    return 0;
}
```

---

9. SWITCH CASE

Used for **multiple choices**.

Program

```c
#include <stdio.h>
int main() {
    int choice;
    printf("Enter choice (1-3): ");
    scanf("%d", &choice);

    switch(choice) {
        case 1: printf("One"); break;
        case 2: printf("Two"); break;
        case 3: printf("Three"); break;
        default: printf("Invalid choice");
    }
    return 0;
}
```

Flowchart

```
 Start
   |
Input choice
   |
Switch(choice)
 /    |    \
1     2     Default
|     |        |
One   Two   Invalid
        |
       End
```

Access Specifier in inheritance (quick notes)

Base member	Public member	Protected inheritance	Private inheritance
public	public	Protected	private
protected	protected	protected	Private
Private	Not accessible	Not accessible	Not accessible

Multilevel inheritance (marks↠ result)

```
#include <iostream>
using namespace std;

// Base class
class Marks {
protected:
    int m1, m2;

public:
    void getMarks(int a, int b) {
        m1 = a;
        m2 = b;
    }

    void showMarks() {
        cout << "Marks 1: " << m1 << endl;
        cout << "Marks 2: " << m2 << endl;
    }
};

// Derived class
class Result : public Marks {
public:
    void displayResult() {
        int total = m1 + m2;
        float percent = total / 2.0;

        showMarks();
```

```
        cout << "Total: " << total << endl;
        cout << "Percentage: " << percent << "%" << endl;
    }
};

int main() {
```